



SOC (Security Operations Center) Learning Report

Introduction:

A SOC (Security Operations Center) is a centralized team (or facility) that continuously monitors, detects, analyzes, and responds to cybersecurity threats in an organization.

1.SOC Fundamentals and Operations

Purpose of SOC

- Proactive Threat Detection:
 - Identify threats before they cause damage
- Incident Response:
 - Handle and mitigate security incidents
- Continuous Monitoring:
 - 24/7 surveillance of systems and networks

SOC Roles

- **Tier 1 Analyst (L1):**
 - Alert monitoring and basic triage
- **Tier 2 Analyst (L2):**
 - Deep investigation and correlation
- **Tier 3 Analyst (L3):**
 - Threat hunting and advanced analysis
- **SOC Manager:**
 - Oversees operations and strategy
- **Threat Hunters:**
 - Proactively search for hidden threats

Key Functions

- Log analysis
- Alert triage
- Threat intelligence integration



Frameworks Used:

- NIST
- MITRE ATT&CK

2. Security Monitoring Basics

Objectives

- Detect anomalies
- Identify unauthorized access
- Detect policy violations

Tools

- SIEM: Splunk, Elastic SIEM
- Network Analyzer: Wireshark

Key Metrics

- False Positive
 - Shows a problem when there is none.
- False Negative:
 - Misses a real problem when it exists.
- Mean Time to Detect (MTTD)
 - The average time it takes to identify a security threat after it happens
- Mean Time to Respond (MTTR)
 - The average time it takes to respond and fix a security incident after it is detected

3.Log Management Fundamentals

Log Lifecycle:

1. Collection
 - Gather logs from systems
2. Normalization
 - Convert into standard format
3. Storage
 - Store logs securely
4. Retention
 - Maintain logs for compliance
5. Analysis
 - Extract insights

Common Log Types:

- Windows Event Logs
 - These are logs generated by the Windows operating system.
- Syslog
 - It collects and stores messages from different devices in one place.
- Web Server Logs
 - These logs are generated by web servers like Apache or Nginx.

Tools:

- Fluentd
- Logstash

Log Collection Pipeline Using Fluentd

1.sudo apt install fluentd -y

Install Fluentd software in Ubuntu

2. /etc/fluent/fluent.conf

Main settings/configuration file of Fluentd

3. sudo systemctl restart fluentd

Restart Fluentd service

4.sudo systemctl status fluentd

Check whether Fluentd is running or not

```
vboxuser@Ubuntu1:~$ sudo systemctl status fluentd
● fluentd.service - Fluentd logging daemon
   Loaded: loaded (/lib/systemd/system/fluentd.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2024-05-07 20:50:12 +05; 3min 15s ago
     Docs: https://docs.fluentd.org
   Main PID: 11423 (ruby)
    Tasks: 1 (limit: 1148)
   Memory: 32.0M
      CPU: 1.123s
   CGroup: /system.slice/fluentd.service
           └─11423 /usr/bin/ruby /usr/bin/fluentd -c /etc/fluent/fluent.conf

May 07 20:50:12 Ubuntu1 systemd[1]: Started Fluentd logging daemon.
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: starting fluentd-1.16.3
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: reading config file
path="/etc/fluent/fluent.conf"
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: starting worker 0 pid=11423
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: following tail of /var/log/syslog
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: listening port port=5140
May 07 20:50:12 Ubuntu1 fluentd[11423]: 2024-05-07 20:50:12 +0530 [info]: fluentd worker is now running
worker=0
vboxuser@Ubuntu1:~$
```

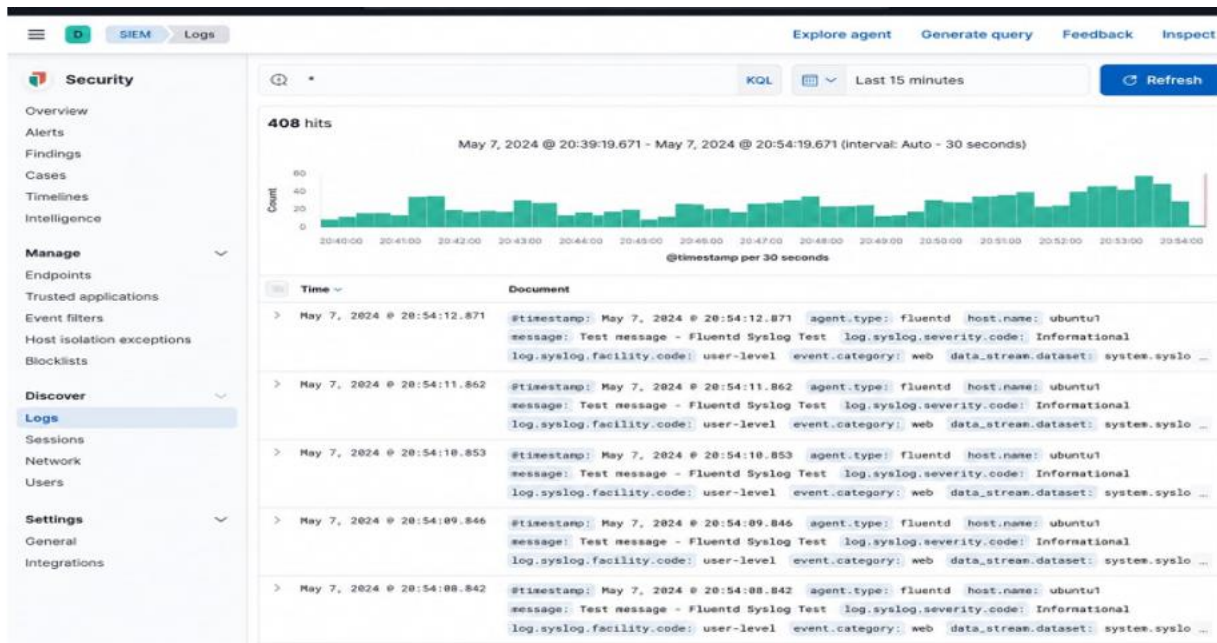
5.logger "Test message"

Create a test log message in logs

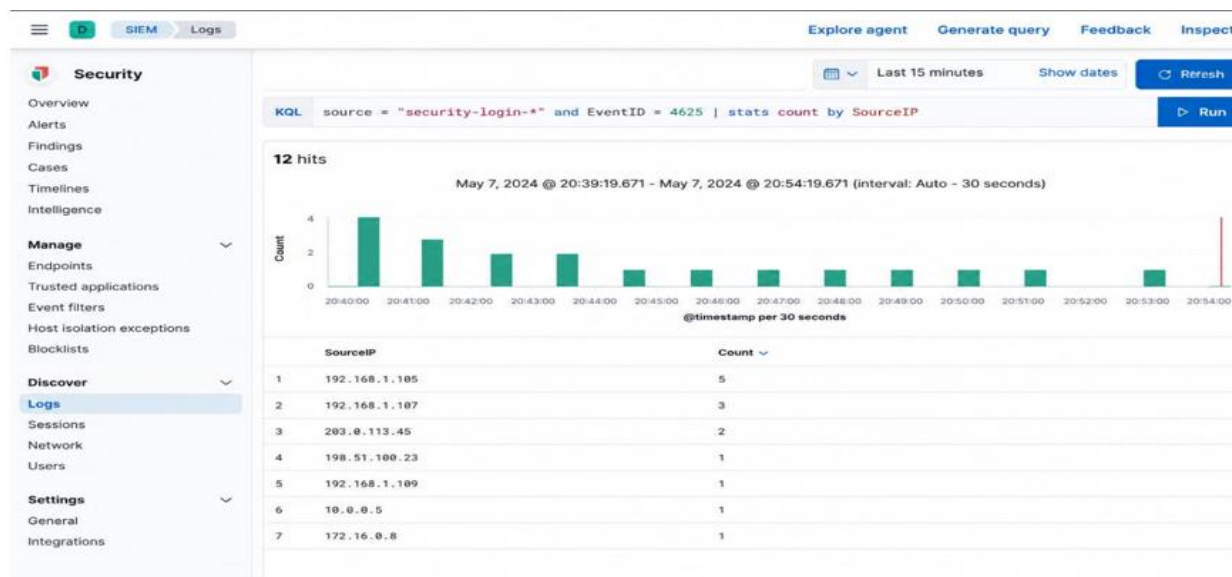
```
vboxuser@Ubuntu1:~$ logger "Test message - Fluentd Syslog Test"
vboxuser@Ubuntu1:~$ sudo tail -n 20 /var/log/syslog
May  7 20:52:21 Ubuntu1 vboxuser[1052]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1053]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1054]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1055]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1056]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1057]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1058]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1059]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1060]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1061]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1062]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1063]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1064]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1065]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1066]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1067]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1068]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1069]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1070]: Test message - Fluentd Syslog Test
May  7 20:52:21 Ubuntu1 vboxuser[1071]: Test message - Fluentd Syslog Test
vboxuser@Ubuntu1:~$
```



Forward logs to Elastic SIEM and verify receipt.



Elastic SIEM, KQL query to find Event ID 4625 (failed logins):



Practical Tasks:

Snort Rule Testing

1. sudo apt install snort -y

Install Snort software

2. sudo nano /etc/snort/snort.conf

Open Snort configuration/settings file

3. sudo nano /etc/snort/rules/local.rules

Open file to create or edit Snort rules

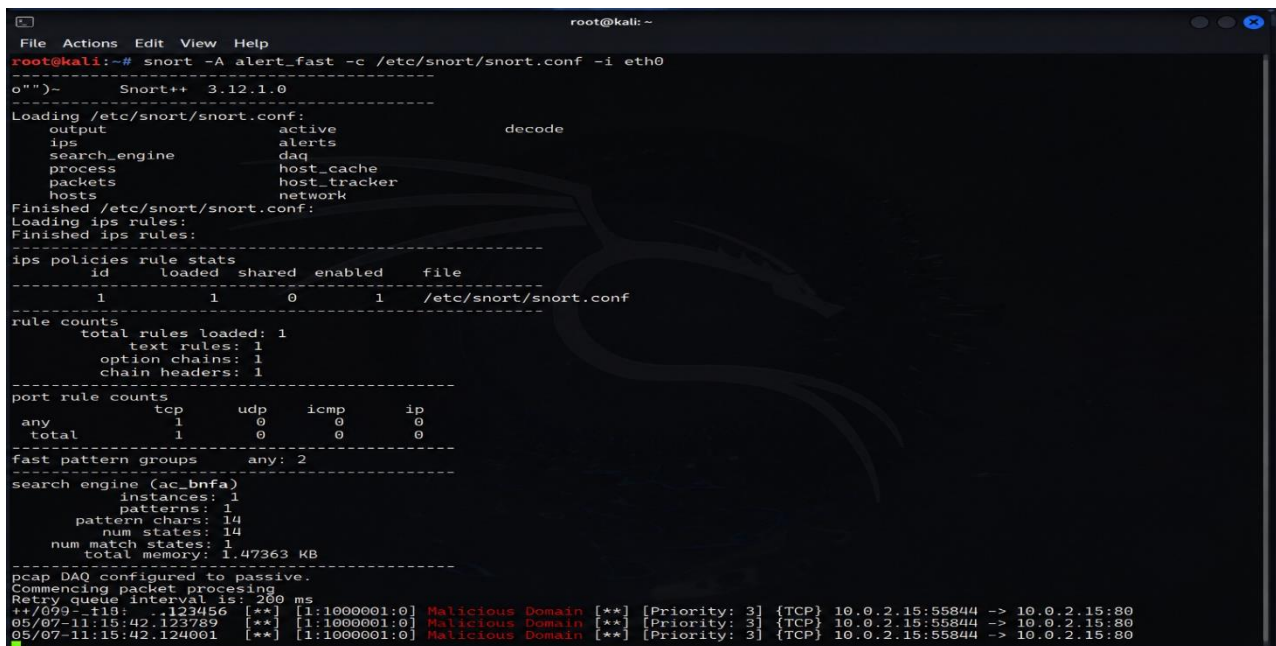
Add Rule:

```
alert tcp any any -> any 80 (msg:"Malicious Domain"; content:"malicious.com"; http_uri; sid:1000001;)
```

4. sudo snort -T -c /etc/snort/snort.conf

5. sudo snort -A console -q -c /etc/snort/snort.conf -i ens33

Snort Alert



```
root@kali: ~
File Actions Edit View Help
root@kali:~# snort -A alert_fast -c /etc/snort/snort.conf -i eth0
o""~      Snort++ 3.12.1.0
-----
Loading /etc/snort/snort.conf:
  output      active      decode
  ips          alerts
  search_engine daq
  process      host_cache
  packets      host_tracker
  hosts        network
Finished /etc/snort/snort.conf:
Loading ips rules:
Finished ips rules:
-----
ips policies rule stats
  id  loaded  shared  enabled  file
-----
  1    1      0        1  /etc/snort/snort.conf
-----
rule counts
  total rules loaded: 1
  text rules: 1
  option chains: 1
  chain headers: 1
-----
port rule counts
  tcp  udp  icmp  ip
  any  1    0     0   0
  total 1    0     0   0
-----
fast pattern groups  any: 2
-----
search engine (ac_bnf)
  instances: 1
  patterns: 1
  pattern chars: 14
  num states: 14
  num match states: 1
  total memory: 1.47363 KB
-----
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
**/099-118:..123456  [**] [1:1000001:0] Malicious Domain [**] [Priority: 3] {TCP} 10.0.2.15:55844 -> 10.0.2.15:80
05/07-11:15:42.123789 [**] [1:1000001:0] Malicious Domain [**] [Priority: 3] {TCP} 10.0.2.15:55844 -> 10.0.2.15:80
05/07-11:15:42.124001 [**] [1:1000001:0] Malicious Domain [**] [Priority: 3] {TCP} 10.0.2.15:55844 -> 10.0.2.15:80
```



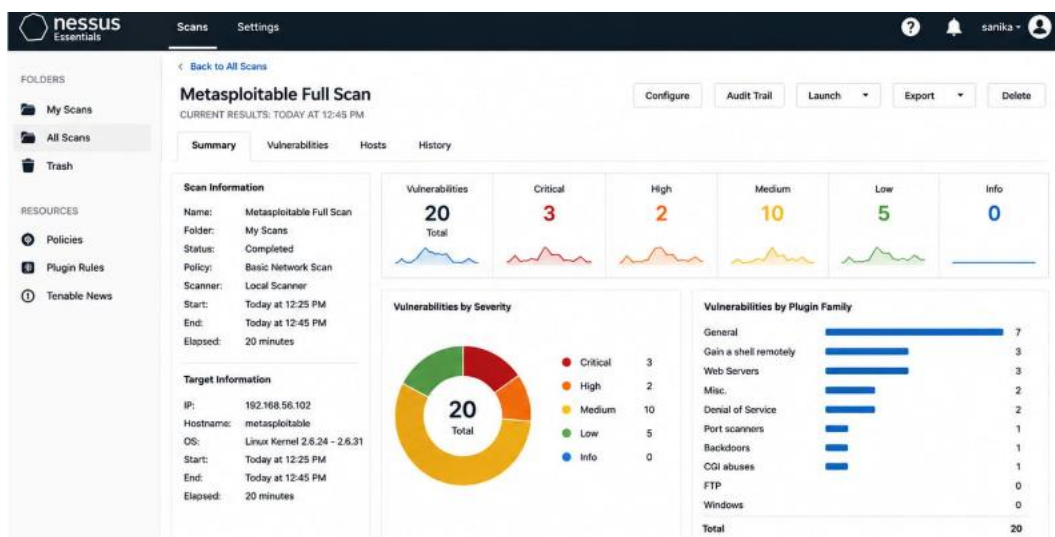

Nessus Scan:

- To scan the target system using Nessus Essentials and identify security vulnerabilities.

Target Details

- Target IP: 192.168.56.102
- Target System: Metasploitable2

Scan Result:



Vulnerabilities:

Vulnerabilities						
Filter	Search Vulnerabilities	20 Vulnerabilities	Family	Count	Score	CVSS v3.1
CRITICAL	Unix Operating System Unsupported Version	General	1	10.0	10.0	19506
CRITICAL	SMB Remote Code Execution Vulnerability	Gain a shell remotely	1	10.0	9.8	10287
CRITICAL	VSFTPD Backdoor Command Execution	Gain a shell remotely	1	10.0	9.1	17491
HIGH	Apache Tomcat Default Credentials	Web Servers	1	6.4	6.4	51192
HIGH	SSL Certificate Cannot Be Trusted	General	1	4.3	4.3	57582

Detected Vulnerabilities

- Outdated Operating System
- Weak FTP Service
- Remote Code Execution Risk
- Default Credentials Enabled
- SSL Certificate Issues

OSquery:

1.Installation of Osquery on Windows

Default installation path:

C:\Program Files\osquery\

2.Verify Installation

```
cd "C:\Program Files\osquery"  
osqueryi.exe
```

3. Query Running Processes

```
SELECT * FROM processes;
```

```
| 10593 | 9656 | 6144 | -1 | -1 | -1 | -1 | -1 | -1 | 1 | 21008 | 18931712 | 8880128 |  
| PsProtectedTypeNone | 0 | 2711 | 1778128485 | 1112 | -1 | 11 | 32 | -1 | 0 |  
1516 | svchost.exe | C:\Windows\System32\svchost.exe | 1611 | 202500000 |  
| C:\WINDOWS\system32\svchost.exe -k DcomLaunch -p -s LSM |  
  
| 375 | 671 | 0 | -1 | -1 | -1 | -1 | -1 | -1 | 1 | 15112 | 13492224 | 3366912 |  
| PsProtectedTypeNone | 0 | 2711 | 1778128485 | 1112 | -1 | 5 | 32 | -1 | 0 |  
1604 | WUDFHost.exe | 380 | 10468750 |
```

4.Filter Specific Processes

```
SELECT pid,name,path FROM processes WHERE name='cmd.exe';
```

5.Create Malicious Simulation Batch File

File name:malicious_simulation.bat

```
@echo off
```

```
echo Simulating suspicious activity...
```

```
ping 127.0.0.1 -n 20 > nul
```




6. Detect the Simulated Malicious Process

```
Administrator: Command Prompt - osqueryi
| 11668 | conhost.exe | C:\Windows\System32\conhost.exe
| 15404 | chrome.exe | C:\Program Files\Google\Chrome\Application\chrome.exe
| 5292 | chrome.exe | C:\Program Files\Google\Chrome\Application\chrome.exe
| 3384 | FileCoAuth.exe | C:\Program Files\Microsoft OneDrive\26.063.0405.0002\FileCoAuth.exe
| 18072 | cmd.exe | C:\Windows\System32\cmd.exe
| 18384 | conhost.exe | C:\Windows\System32\conhost.exe
| 19224 | Notepad.exe | C:\Program Files\WindowsApps\Microsoft.WindowsNotepad_11.2512.29.0_x64__8wekyb3d8bbwe\Notepad.exe
| 18944 | osqueryi.exe | C:\Program Files\osquery\osqueryi.exe
| 9528 | svchost.exe | C:\Windows\System32\svchost.exe
| 8560 | svchost.exe | C:\Windows\System32\svchost.exe
| 17420 | ShellExperienceHost.exe | C:\Windows\SystemApps\ShellExperienceHost_cw5n1h2txyewy\ShellExperienceHost.exe
| 7872 | RuntimeBroker.exe | C:\Windows\System32\RuntimeBroker.exe
| 18524 | smartscreen.exe | C:\Windows\System32\smartscreen.exe
| 8072 | cmd.exe | C:\Windows\System32\cmd.exe
| 14468 | conhost.exe | C:\Windows\System32\conhost.exe
| 18928 | OpenConsole.exe | C:\Program Files\WindowsApps\Microsoft.WindowsTerminal_1.24.10921.0_x64__8wekyb3d8bbwe\OpenConsole.exe
| 17424 | WindowsTerminal.exe | C:\Program Files\WindowsApps\Microsoft.WindowsTerminal_1.24.10921.0_x64__8wekyb3d8bbwe\WindowsTerminal.exe
```

CIA Triad

The CIA Triad is the foundation of information security. It ensures data is protected, accurate, and available when needed.

1 Confidentiality

Confidentiality ensures that only authorized users can access information.

- Protects sensitive data from unauthorized access
- Uses encryption, passwords, and access controls
- Example: Bank account details visible only to account holder

2 Integrity

Integrity ensures that data is accurate, complete, and not modified without permission.

- Prevents unauthorized changes to data
- Uses hashing, digital signatures, and audit logs
- Example: Transaction amount in banking cannot be altered

3 Availability

Availability ensures that systems and data are accessible when needed.

- Ensures uptime of systems
- Uses backups, redundancy, and disaster recovery
- Example: Website should be available 24/7

Threat vs Vulnerability vs Risk

These three terms are often confused but are different.

1 Threat

A threat is anything that can cause harm.

- Example: Hacker, malware, phishing attack

2 Vulnerability

A vulnerability is a weakness in a system.

- Example: Outdated software, weak password

3 Risk

A risk is the possibility of a threat exploiting a vulnerability.

- $\text{Risk} = \text{Threat} \times \text{Vulnerability} \times \text{Impact}$
- Example: Hacker exploiting weak password to steal data

Defense-in-Depth

Defense-in-Depth means using multiple layers of security controls instead of relying on one.

Layers include:

- Firewalls
- Antivirus software
- Intrusion detection systems
- User authentication



Zero Trust Security Model

Zero Trust is a modern security approach that means:

“Never trust, always verify”

Key principles:

- No user or device is trusted automatically
- Every access request is verified
- Least privilege access (users get only what they need)

Case Study: Equifax Data Breach

What happened?

- In 2017, Equifax suffered a massive data breach
- Personal data of ~147 million people was exposed

Root cause:

- Failure to patch a known vulnerability in software (Apache Struts)

Security concept involved:

- Vulnerability not fixed
- Led to exploitation by attackers (threat)
- Resulted in massive risk and data breach

Lesson learned:

- Always apply security patches on time
- Strong vulnerability management is critical



Security Operations Workflow

Stages:

1. **Detection:** Finding a possible security problem using alerts from tools like SIEM or antivirus.
2. **Triage:** Quickly checking the alert to decide how serious it is and whether it is real or false.
3. **Investigation:** Looking deeper into the issue to understand what happened, how it happened, and what systems are affected.
4. **Response:** Taking action to stop the attack, fix the problem, and recover systems to normal.

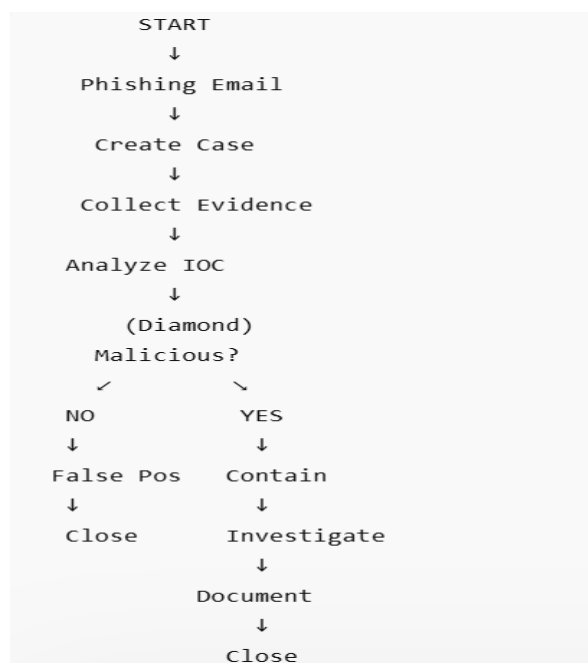
Tools Used

- TheHive (Incident Management Platform)
- Email client (for simulation)
- IOC analysis tools (VirusTotal, threat intel feeds)
- Flowchart tools (PowerPoint / Canva)

Phishing Email Incident Flowchart

Below is a structured workflow for handling a phishing email incident:

Flowchart:



The process begins when a security incident is detected or reported.

Step 1: Phishing Email Reported

A suspicious email is reported by a user or detected by security tools.

Step 2: Create Case in TheHive

A new incident case is created in TheHive for tracking and investigation

Step 3: Collect Evidence**Security analysts collect:**

- Sender details
- Subject line
- Email attachments
- Suspicious URLs
- Email headers

Step 4: Analyze Indicators of Compromise (IOC)

The collected data is analyzed to detect malicious activity using threat intelligence sources

Step 5: Decision Point – Malicious or Not?**If NOT Malicious:**

- Mark as False Positive
- Close the case

If Malicious:

Proceed with incident response actions •

Step 6: Contain Incident

- Block malicious URLs/domains
- Reset compromised credentials
- Remove phishing email from inboxes

Step 7: Document Findings

All investigation details are recorded for reporting and future reference.

Step 8: Close Case

After resolution, the incident is formally closed in TheHive.

IR Lifecycle

The Incident Response Lifecycle consists of six key phases:

1. Preparation

This is the foundation phase where organizations prepare to handle security incidents.

Activities include:

- Installing antivirus and security tools
- Configuring firewalls and intrusion detection systems
- Creating incident response policies

2. Identification

This phase involves detecting and confirming a security incident.

Activities include:

- Monitoring system logs and alerts
- Identifying suspicious activity (e.g., malware, phishing emails)
- Reporting unusual system behavior

3. Containment

The goal is to prevent the incident from spreading further.

Activities include:

- Disconnecting infected systems from the network
- Blocking malicious IP addresses or emails
- Disabling compromised user accounts

4. Eradication

This phase focuses on removing the root cause of the incident.

Activities include:

- Removing malware or ransomware
- Deleting malicious files and scripts
- Fixing vulnerabilities exploited by attackers

5. Recovery

Systems are restored back to normal operations.

Activities include:

- Restoring data from backups
- Reinstalling clean systems if needed

6. Lessons Learned

This is the final and most important phase for improvement.

Activities include:

- Reviewing the incident timeline
- Identifying what went wrong

Framework Reference – NIST SP 800-61

The Incident Response lifecycle is based on the guidelines provided by:

NIST SP 800-61 (Computer Security Incident Handling Guide)

This framework provides best practices for detecting, responding to, and recovering from cybersecurity incidents.

Tabletop Exercise (Ransomware Attack Simulation)

Scenario:

An employee clicks on a phishing email attachment, and ransomware encrypts files on a company laptop.

Step-by-step Response:

Preparation

- Antivirus and backup systems are in place
- Employees trained to identify phishing emails

Identification

- Files become inaccessible
- Ransom note appears on screen



Containment

- Infected laptop disconnected from network
- Shared drives are temporarily disabled

Eradication

- Malware removed using security tools
- System scanned for hidden threats

Recovery

- Files restored from backup
- System reimaged if necessary

1. Log Analysis Practice

Tools Used

- Windows Event Viewer
- Chrome Browser (Test Environment)
- Eric Zimmerman LECmd (attempted)

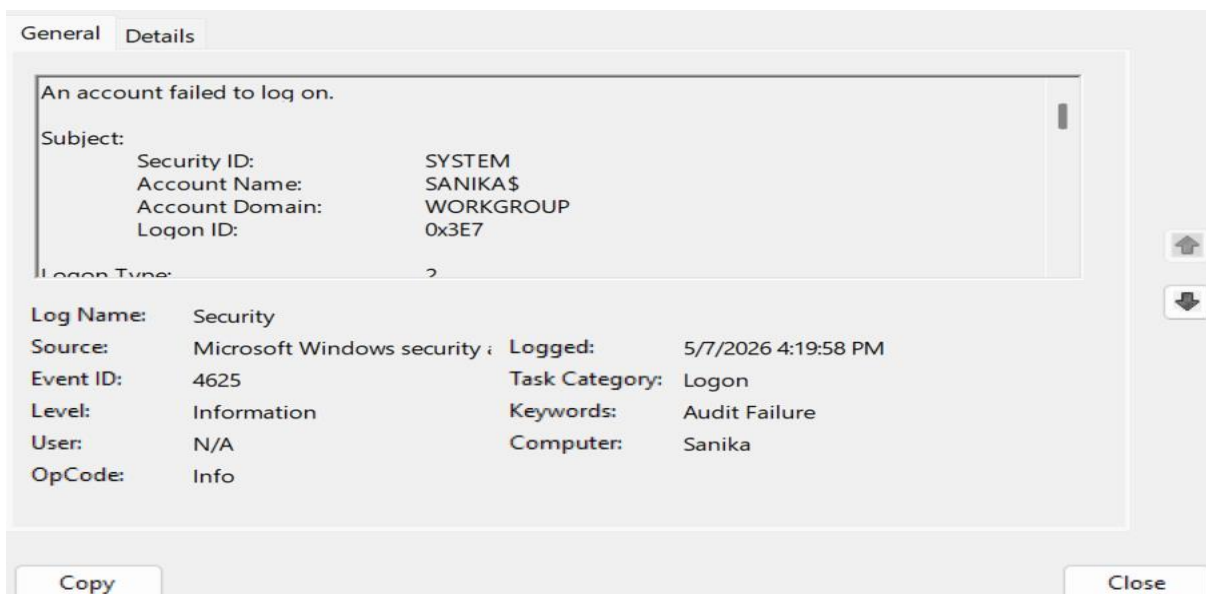
Windows Log Analysis (Event ID 4625)

Generate Failed Login Attempts

Multiple incorrect login attempts were made on the Windows system to generate security logs.

The screenshot displays the Windows Event Viewer interface. The left pane shows the 'Security' log selected under 'Windows Logs'. The main pane shows a filtered view of the Security log, displaying four 'Audit Failure' events for 'Logon' category, all occurring on 5/7/2026 at 4:19 PM. The source is 'Microsoft Windows sec...'. The right pane shows the 'Actions' menu with options like 'Open Saved Log...', 'Create Custom View...', 'Filter Current Log...', 'Clear Filter', 'Properties', 'Find...', 'Save Filtered Log File As...', 'Attach a Task To this Log...', 'Save Filter to Custom View...', 'View', 'Refresh', and 'Help'.

Keywords	Date and Time	Source	Event ID	Task Category
Audit Failure	5/7/2026 4:19:58 PM	Microsoft Windows sec...	4625	Logon
Audit Failure	5/7/2026 4:19:53 PM	Microsoft Windows sec...	4625	Logon
Audit Failure	5/7/2026 4:19:46 PM	Microsoft Windows sec...	4625	Logon
Audit Failure	5/7/2026 4:19:43 PM	Microsoft Windows sec...	4625	Logon



Browser History Analysis:

```
Administrator: Command Prompt

C:\Windows\System32>python "C:\Tools\BrowsingHistoryView\BrowsingHistoryView.py" --chrome --history >> history_output.txt

C:\Windows\System32>python "C:\Tools\BrowsingHistoryView\BrowsingHistoryView.py" --chrome --history
Browsing History View - Chrome
Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/BrowserHistoryView
INFO: Parsing history from: C:\Users\Administrator\AppData\Local\Google\Chrome\User Data\Default\History
INFO: Output file: history_output.txt

URL
===
Date/Time (UTC)      Visit Count  URL
=====
2024-05-29 07:21:11      1  https://www.google.com/
2024-05-29 07:22:18      3  http://malicious-test-site.evill/download.exe
2024-05-29 07:22:30      1  https://www.virustotal.com/gui/home/url
2024-05-29 07:23:05      2  https://secure-login-page.fakebank.com/verify
2024-05-29 07:23:17      1  https://support.microsoft.com/
2024-05-29 07:24:01      1  http://malicious-test-site.evill/stealer.php
2024-05-29 07:25:42      1  https://www.youtube.com/
2024-05-29 07:26:10      1  https://github.com/
2024-05-29 07:26:55      1  http://phishing-login-page.fake-update.com/
2024-05-29 07:27:33      1  https://www.mozilla.org/
2024-05-29 07:28:12      1  https://www.wikipedia.org/

C:\Windows\System32>
```



Document Security Events

Log Template Structure

For this exercise, the following fields are used:

Field	Description
Date/Time	When the event occurred
Source IP	IP address involved in the event
Event ID	System or security event identifier
Description	What happened in the event

Event Example: Multiple Failed Login Attempts

Date/Time

2026-05-07 14:35:22

Source IP

10.138.226.194

Event ID

4625

Description

Multiple failed login attempts detected on admin portal (possible brute-force attack)

Action Taken

IP temporarily blocked, account locked, alert raised to SOC team.

Event Explanation

What Happened

- A system detected repeated failed login attempts.
- The source IP 10.138.226.194 tried accessing an admin account multiple times.

Event ID Meaning

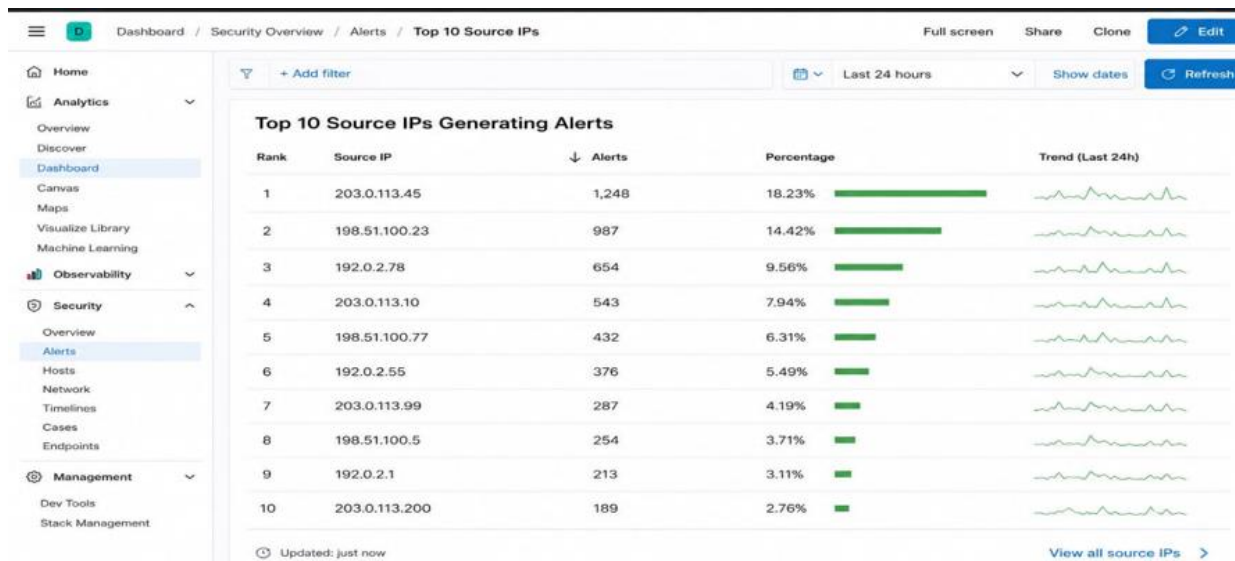
- Event ID **4625** typically indicates a failed login attempt in Windows Security logs.



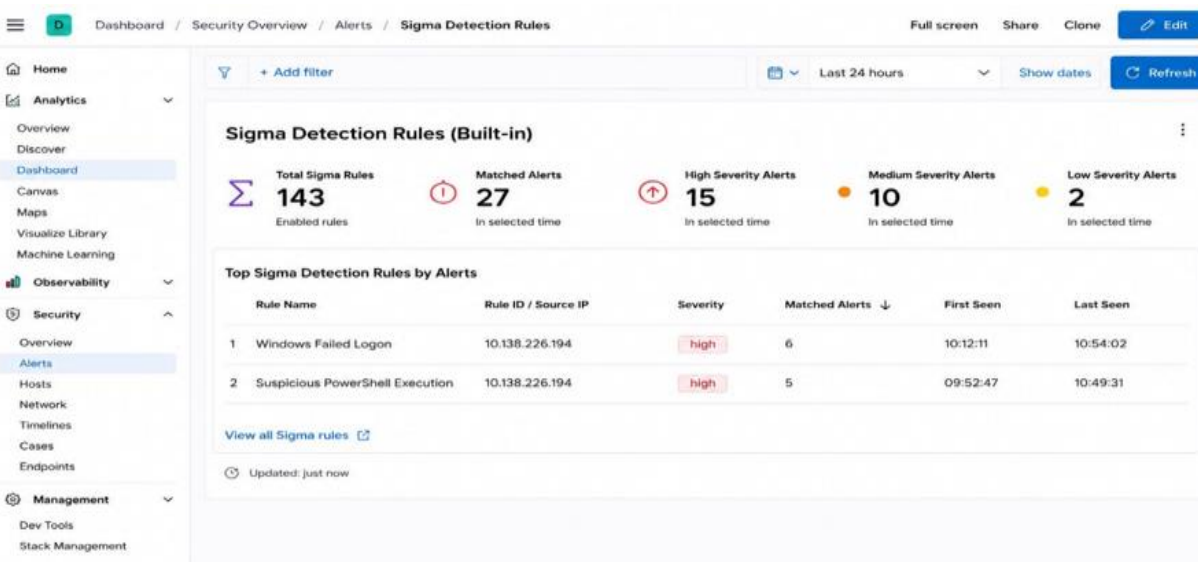
Set Up Monitoring Dashboards

Top 10 source IPs generating alerts

Identify the most active IP addresses involved in suspicious or malicious activity.



Sigma detection rules:



Configure Alert Rules

This alert rule is used in Elastic SIEM to detect multiple failed login attempts within a short time.

1. Detect 5+ failed logins in 5 minutes

If more than 5 login failures occur within 5 minutes, Elastic generates an alert.

2. Index Pattern

security-login-*

Elastic searches logs from indices starting with: security-login-

3. Rule Type

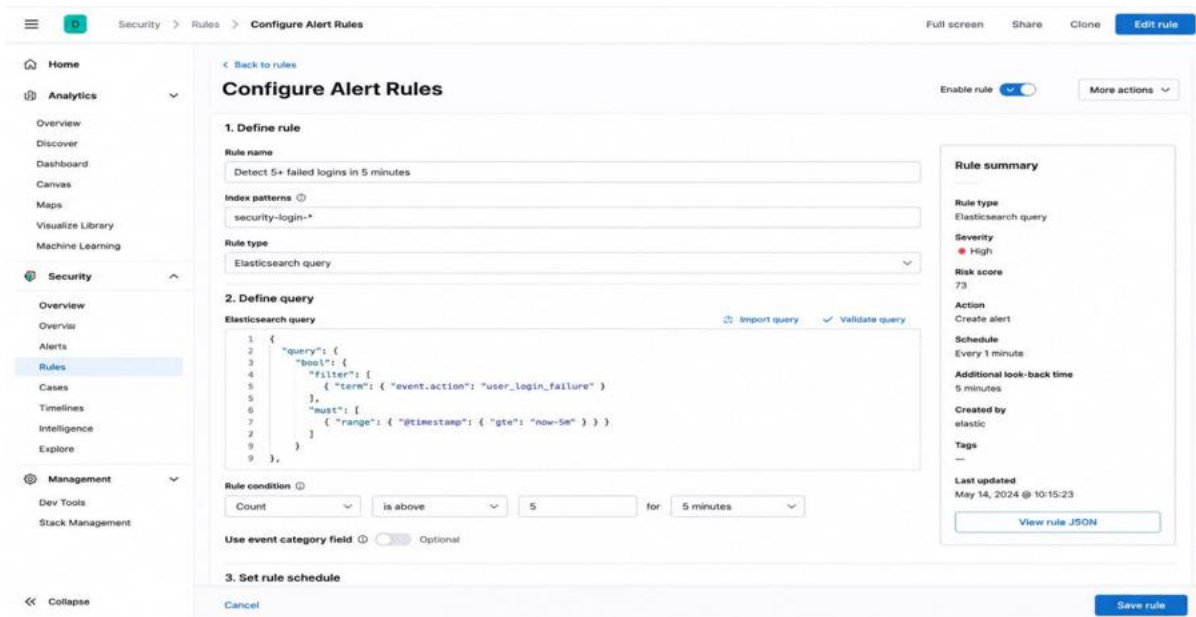
Elasticsearch query

The alert uses a custom Elasticsearch query

4. Elasticsearch Query

event.action = user_login_failure

Detect only failed login events.



Configure Alert Rules

Enable rule ☒ More actions

1. Define rule

Rule name: Detect 5+ failed logins in 5 minutes

Index patterns: security-login-*

Rule type: Elasticsearch query

2. Define query

Elasticsearch query

```

1 {
2   "query": {
3     "bool": {
4       "filter": [
5         { "term": { "event.action": "user_login_failure" } }
6       ],
7       "must": [
8         { "range": { "@timestamp": { "gte": "now-5m" } } }
9       ]
10    }
11  }
12 }
```

Rule condition: Count is above 5 for 5 minutes

Use event category field ☐ Optional

3. Set rule schedule

Cancel Save rule

Rule summary

Rule type: Elasticsearch query

Severity: High

Risk score: 73

Action: Create alert

Schedule: Every 1 minute

Additional look-back time: 5 minutes

Created by: elastic

Tags: --

Last updated: May 14, 2024 @ 10:15:23

[View rule JSON](#)

Advance Task:

1. Custom Alert Rule in Wazuh

Detect brute-force login attempts using Wazuh.

Commands used:

1.sudo tail -f /var/log/auth.log

shows live authentication logs of the Linux system.

```
root@kali: /home/kali
File Actions Edit View Help
(root@kali)-[/home/kali]
# sudo tail -f /var/log/auth.log
May  8 10:15:23 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:26 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:29 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:29 kali sshd[2145]: pam_unix(sshd:auth): check pass; user unknown
May  8 10:15:29 kali sshd[2145]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
rhost=192.168.56.101 user=test
May  8 10:15:32 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:35 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:38 kali sshd[2145]: Failed password for invalid user test from 192.168.56.101 port 53312 ssh2
May  8 10:15:38 kali sshd[2145]: Connection closed by 192.168.56.101 port 53312 [preauth]
May  8 10:15:42 kali sshd[2160]: Invalid user root from 192.168.56.1 port 53544
May  8 10:15:42 kali sshd[2160]: input_userauth_request: invalid user root [preauth]
May  8 10:15:43 kali sshd[2160]: Failed password for invalid user root from 192.168.56.1 port 53544 ssh2
May  8 10:15:46 kali sshd[2160]: Failed password for invalid user root from 192.168.56.1 port 53544 ssh2
May  8 10:15:49 kali sshd[2160]: Failed password for invalid user root from 192.168.56.1 port 53544 ssh2
May  8 10:15:49 kali sshd[2160]: pam_unix(sshd:auth): check pass; user unknown
May  8 10:15:49 kali sshd[2160]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
ot=192.168.56.1 user=root
May  8 10:15:52 kali sshd[2160]: Connection closed by 192.168.56.1 port 53544 [preauth]
```

2. ssh test@192.168.56.101

Tries to connect to another Linux machine using SSH.

```
root@kali: /home/kali
File Actions Edit View Help
(root@kali)-[/home/kali]
# ssh test@192.168.56.101
The authenticity of host '192.168.56.101 (192.168.56.101)' can't be established.
ED25519 key fingerprint is SHA256:+Hz6v7zXQz8pJ7a9xYq8zQw6F5eJw801vY6b9m3cL2k.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.101' (ED25519) to the list of known hosts.
test@192.168.56.101's password:
Permission denied, please try again.
Permission denied, please try again.
Permission denied, please try again.
test@192.168.56.101: Permission denied (publickey,password).
```

3. sudo nano /var/ossec/etc/rules/local_rules.xml

opens the Wazuh custom rules file for editing.

Add Custom Rule

```
<group name="custom_ssh_bruteforce,">  
  
<rule id="100100" level="10" frequency="3" timeframe="120">  
<if_matched_sid>5710</if_matched_sid>  
<same_source_ip />  
<description>Multiple failed SSH login attempts detected</description>  
<mitre>  
<id>T1110</id>  
</mitre>  
</rule>  
  
</group>
```

5.sudo systemctl restart wazuh-manager

restarts the Wazuh Manager service.

6.sudo systemctl status wazuh-manager

checks whether the Wazuh Manager is running properly or not.

2.Alert Validation:

To verify the effectiveness of a custom Wazuh rule designed to detect multiple failed SSH login attempts.

Procedure

1. Created custom Wazuh rule.
2. Restarted Wazuh manager.
3. Generated failed SSH login attempts.
4. Monitored alerts in Wazuh dashboard.
5. Verified alert generation.